EE 4370 Lab 5

Part 1

For this experiment we are learning how to use an interrupt thread. We increment our event counter every time the button is pressed. Then the pending counter is decreased every time the interrupt is called. In my observations I was unable to get outputs which could show the outputs clearly. I will be referring to the outputs which Professor Pedram allowed us to use from his simulation. In the outputs shown in the image we can see the part without errors are circled in blue. This is correct because the values of the pending counter match the event counter. Since we know the pending counter increases by 1 every time and the event counter only go up. We can make the table and figure out the push and loop at each event. For example, in the example we have 7 events happen from 9 to 16 and 1 gets resolved at 16. This leaves us with a print value of 16 events and 6 pending events. An example of the output being incorrect is shown circled in red. The output is very difficult to detect but makes sense to why it is wrong. Shown below event counter goes from 27 to 30 and pending event goes from 1 to 2. However, 3 more events occur between 27 and 30, plus we have one at 27, then one gets resolved at 30. Meaning we should have 3 pending events at event 30. That is an example of error in the output.

AI Thinker ESP32-CAM

EE_4370_Lab_5.ino

```
const byte IN_GPIO = 16;
const byte LED_GPIO = 12;

volatile int PendingCounter; // When it is not 0 we take reading from sensor
volatile int EventCounter = 0;

void IRAM_ATTR onSwitch() {
  PendingCounter++; // Increment by 1
  EventCounter++; // Increment by 1
}

void setup() {
  Serial.begin(115200);
  pinMode(IN_GPIO, INPUT_PULLUP);
  attachInterrupt(IN_GPIO, onSwitch, FALLING); // Calls interrupt on switch being pressed
}

void loop() {
  if (PendingCounter > 0) {
    Serial.print(EventCounter); // Prints value
    Serial.print("  ");

    PendingCounter--;
    Serial.println(PendingCounter); // Prints value

    toggleLED(); // Access the sensor to print values
  }
}

void toggleLED() {
  digitalWrite(LED_GPIO, HIGH);
  delay(50);
  digitalWrite(LED_GPIO, LOW);
}

```

Outputs

Unlocked no delay	
Event Counter	Pending Counter
1	0
2	0
3	0
4	1
5	0
6	0
8	1
8	0
9	0
16	6
19	8
19	7
19	6
19	5
20	5
20	4
20	3
21	3
21	2
21	1
22	1
22	0
24	1
24	0
25	0
26	0
27	1
30	2
31	2
31	1
31	0
32	0
33	0

Part 2

In this part we are simulating the same thing, but we are introducing a lock which will protect the values of Pending Counter. As shown in the code below we have a function called switchMux which will protect the values of the variable. In the ISR function we lock it then change the counters and then release the lock. We also do the same thing in the main loop. Doing this prevents our output from having any errors. Once I simulated my circuit I had zero problems with the output. As shown below I proved for part of my output why it is correct and makes logical sense. This is only one solution to solve this problem and there could be easier ways.

Code

AI Thinker ESP32-CAM

EE_4370_Lab_5.ino

```
38  const byte IN_GPIO = 16;
39  const byte LED_GPIO = 12;
40
41  volatile int PendingCounter; // When it is not 0 we take reading from sensor
42  volatile int EventCounter = 0;
43
44  portMUX_TYPE switchMux = portMUX_INITIALIZER_UNLOCKED; // Name of Lock
45
46  void IRAM_ATTR onSwitch() {
47    portENTER_CRITICAL_ISR(&switchMux); // Lock
48    PendingCounter++; // Increment by 1 and Critical Section
49    EventCounter++; // Increment by 1 and Critical Section
50    portEXIT_CRITICAL_ISR(&switchMux); // Release
51  }
52
53  void setup() {
54    Serial.begin(115200);
55    pinMode(IN_GPIO, INPUT_PULLUP);
56    attachInterrupt(IN_GPIO, onSwitch, FALLING); // Calls interrupt on switch being pressed
57  }
58
59  void loop() {
60    if (PendingCounter > 0) {
61      Serial.print(EventCounter); // Prints value
62      Serial.print("  ");
63      portENTER_CRITICAL(&switchMux); // Lock
64      PendingCounter--; // Critical Section
65      Serial.println(PendingCounter); // Prints value
66      portEXIT_CRITICAL(&switchMux); // Release
67
68      toggleLED(); // Access the sensor to print values
69    }
70  }
71
72  void toggleLED() {
73    digitalWrite(LED_GPIO, HIGH);
74    delay(50);
75    digitalWrite(LED_GPIO, LOW);
76  }
77
```

Output

```
1  1
2  0
3  0
4  0
7  2
7  1
13  6
13  5
13  4
16  6
16  5
18  6
22  9
22  8
23  8
24  8
24  7
25  7
25  6
27  7
27  6
32  10
32  9
32  8
32  7
32  6
32  5
32  4
32  3
32  2
32  1
32  0
33  0
36  2
37  2
37  1
41  4
41  3
41  2
```

```
7  2
7  1
13  6
13  5
13  4
16  6
16  5
18  6
22  9
22  8
23  8
24  8
24  7
```

Push	EC = 7	PC = 2	Print
Loop	EC = 7	PC = 1	Print
Push	EC = 8	PC = 2	
Push	EC = 9	PC = 3	
Push	EC = 10	PC = 4	
Push	EC = 11	PC = 5	
Push	EC = 12	PC = 6	
Push	EC = 13	PC = 6	Print
Loop	EC = 13	PC = 5	Print
Loop	EC = 13	PC = 4	Print
Push	EC = 14	PC = 5	
Push	EC = 15	PC = 6	
Push	EC = 16	PC = 6	Print
Loop	EC = 16	PC = 5	Print

(keeps going)